

LECTURE 11

Data Modeling and Database Design

From a fuzzy business conversation to precise tables, columns, and keys

CSC270 · Database Systems

COMSATS University Islamabad, Lahore Campus

What This Lecture Covers

- 1 Why data modeling comes before building anything, not after
- 2 Conceptual data models — capturing meaning, independent of technology
- 3 Logical data models — adding relational structure, still DBMS-independent
- 4 Relational database design as this course's specific focus
- 5 Mapping conceptual → logical → relational schema
- 6 Practical considerations that separate a good design from a merely working one

Why Model Before You Build

- A data model is a shared vocabulary for data, its relationships, and its constraints — letting a designer, a client, and a DBMS agree on what “a branch” or “a viewing” means before any SQL is written.
- Tables are a technology-specific answer to a business question — separating “what does the business need?” from “how do we implement it?” lets you get the meaning right first, cheaply.

WARNING

The cost of getting this wrong compounds: a missing requirement caught during modeling costs a conversation and a redrawn diagram. The same requirement, caught after production data depends on the shape, can cost a multi-week migration.

Conceptual → Logical → Relational Schema

Conceptual model

“What are the things, and how do they relate?” — technology-independent, for business stakeholders



Logical model

“What tables, columns, and keys does this need?” — DBMS-product-independent, for designers



Relational schema

“What does CREATE TABLE actually say?” — concrete data types and DBMS-specific detail

Conceptual Data Models

- A technology-independent description of an organization's data: the entities (things to track), relationships (connections), and attributes (properties).
- No mention of tables, columns, data types, or which DBMS will run it.
- The Entity-Relationship (ER) model, next lecture's subject, is the standard notation for building conceptual models.

“A branch employs several staff.” “A client may view several properties, and a property may be viewed by several clients.” “Each property is owned by exactly one private owner.”

TIP

A conceptual model should survive a change of DBMS unchanged. If moving from PostgreSQL to Oracle forces you to redraw it, the model was capturing implementation detail, not business meaning.

Turning a Conversation into a Conceptual Model

- Staff say “a branch employs several staff”: entities Branch and Staff, joined by one relationship Has (1:N).
- Staff say “a client may view several properties”: entities Client and PropertyForRent, joined by Views (M:N).
- Staff say “each property is owned by exactly one private owner”: entities PrivateOwner and PropertyForRent, joined by Owns (1:N).

Nouns become entity types (Branch, Staff, Client, PropertyForRent, PrivateOwner); verbs become relationships (Has, Views, Owns); facts about each thing become attributes (staffNo, salary, rent).

TIP

A conceptual model should survive a change of DBMS unchanged. If moving from PostgreSQL to Oracle forces you to redraw it, the model was capturing implementation detail, not business meaning.

Logical Data Models

- Takes the entities, relationships, and attributes from the conceptual model and organizes them according to the rules of a particular family of data model — relational, hierarchical, network, or object-oriented.
- This course works entirely within the relational family: a logical model means a complete set of relation schemas — names, attributes, primary keys, foreign keys — with no data types or storage details yet.
- Answers “what relations do we need, and how do they reference each other?” — not “is salary a DECIMAL(8,2) or an INT?”

Three Properties of a Good Design

1

Correctness

The design faithfully represents every entity, relationship, and business rule the organization actually has — no more, no less.

2

Non-redundancy

The same fact is not stored in more than one place, so it can never drift out of sync (the motivation for normalization, Unit 5).

3

Robustness under real use

The design supports the queries, transactions, and volume of data the organization will actually throw at it.

WARNING

Relational database design is not the same as “writing SQL.” Syntactically perfect CREATE TABLE statements can still represent the business incorrectly — SQL syntax accepts it without complaint; design discipline catches it anyway.

Redundancy in Practice: An Example

1

Redundant copy

PropertyForRent also stores the owner's name and address next to ownerNo, for convenience.

2

Update anomaly

Change the owner's address in one row and the copies in other rows silently go stale.

3

Silent inconsistency

Two rows for the same owner can now disagree, and no constraint in the schema will complain.

WARNING

Redundancy is more than untidy: it is the root cause of update, insert and delete anomalies. Normalization in Unit 5 removes it systematically.

What Goes Wrong: An Update Anomaly

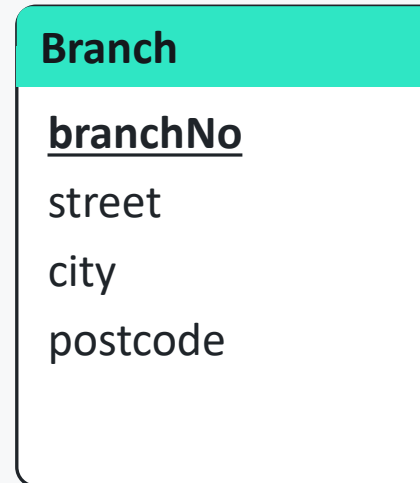
EXAMPLE

Suppose PropertyForRent stores ownerName and ownerAddress. Owner CO46 owns PA14 and PG4, so the same address is written into two rows.

Now the owner moves. Updating only PA14 leaves PG4 with the old address: two rows disagree about one fact, and nothing in the schema complained.

Conceptual Model → Logical Model

- Every strong entity type becomes a relation; the entity's identifying attribute becomes the primary key.
- Every simple attribute becomes a column of that relation.
- Every relationship type becomes a foreign key (1:1, 1:N) or an entirely new relation (M:N) — the precise rule depends on cardinality, covered fully in Lecture 12.



Conceptual — an ER entity type

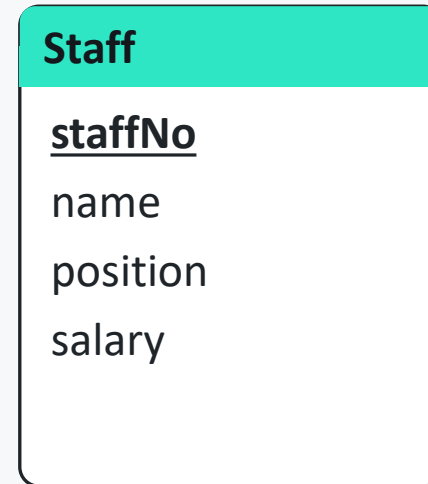
Logical — the mapped relation

branchNo	street	city	postcode
B003	163 Main St	Lahore	54000
B005	22 Deer Rd	Karachi	74200

Nothing was lost — every attribute became a column and the identifying attribute became the primary key — but nothing was added either: no data type, no index, no storage engine.

Mapping Staff: Entity to Relation

- Staff is a strong entity: staffNo identifies each occurrence on its own, so staffNo becomes the primary key.
- Its simple attributes name, position, and salary each become a column of the Staff relation.
- The Has relationship (1:N to Branch) becomes a foreign key, branchNo, placed on the Staff side.



Conceptual — an ER entity type

Logical — the mapped relation

staffNo	name	position	salary
SL21	John White	Manager	30000
SG37	Ann Beech	Assistant	12000

The foreign key branchNo is left out of this table to keep it to the entity's own attributes; it is added once the Has relationship is mapped.

Logical Model → Relational Schema

- Concrete data types for every attribute — DECIMAL(8,2), INTEGER, or NUMERIC?
- Column-level constraints — NOT NULL, CHECK constraints enforcing domain rules.
- Indexing strategy — primary keys and foreign keys, at minimum.
- Naming conventions and DBMS-specific CREATE TABLE syntax.

```
CREATE TABLE Branch (  
    branchNo  CHAR(4)      NOT NULL,  
    street    VARCHAR(40)  NOT NULL,  
    city      VARCHAR(30)  NOT NULL,  
    postcode  CHAR(5)      NOT NULL,  
    PRIMARY KEY (branchNo)  
);
```

NOTE

Connolly's textbook calls this final translation step physical database design — unlike the logical model, it is allowed, even expected, to differ between DBMS products and trade conceptual purity for performance.

Relational Schema for Staff

- Concrete data types: staffNo CHAR(4), name VARCHAR(40), salary DECIMAL(8,2).
- NOT NULL on every column that must always hold a value.
- CHECK (salary > 0) enforces a domain rule from Lecture 6.
- A foreign key on branchNo references Branch, so every staff row points at a real branch.

```
CREATE TABLE Staff (  
    staffNo CHAR(4) NOT NULL,  
    name VARCHAR(40) NOT NULL,  
    salary DECIMAL(8,2) NOT NULL,  
    CHECK (salary > 0),  
    branchNo CHAR(4) NOT NULL,  
    PRIMARY KEY (staffNo),  
    FOREIGN KEY (branchNo) REFERENCES  
    Branch(branchNo) );
```

NOTE

The CHECK and FOREIGN KEY lines enforce rules the business already has; they change how the rules are stored, never what they mean.

Relational Database Design Considerations

Consideration	Question to Ask
Redundancy	Is any fact stored in more than one place?
Update anomalies	Can inserting, updating, or deleting one row corrupt or lose other facts?
Appropriate keys	Is the primary key stable and minimal?
Referential integrity	Does every foreign key correctly reference a real, enforced primary key?
Growth and volume	Will this design still perform acceptably at 100x the current data volume?
Security and access	Should every user see every column (e.g. salary)?
Naming consistency	Are attribute names consistent across every relation that shares a domain?

Applying the Checklist to Staff

CHECK

“A Staff member has a staffNo, a name, a position, and a salary, and works at exactly one branch.” Every attribute belongs to that one thing, so the sentence test passes.

Redundancy check: Staff does not repeat branch name or address. Those live only in Branch, so a branch move changes one row, not every staff row.

A Cheap Sanity Check

TIP

For every relation, read its name and attributes as a sentence: “A Branch has a branchNo, a street, a city, and a postcode.” If the sentence sounds natural and every attribute clearly belongs to that one thing, the relation is probably well-formed.

If you find yourself saying “...and also the name and address of whoever manages it” for a relation that isn't Staff, that's redundancy trying to sneak in — exactly what Unit 5's normalization theory formalizes and eliminates.

Your Turn: Does This Relation Pass?

TRY IT

Relation: Viewing (clientNo, propertyNo, viewDate, comment). Read it as a sentence and decide whether each attribute belongs to the viewing event. Pause before the next slide.

Hint: a viewing is an event between one client and one property, so viewDate and comment describe the pairing. That is why they belong in Viewing, not in Client.

The Pipeline, Step by Step

Conceptual model

Staff say “a branch employs several staff”: entities and relationships, no tables yet.



Logical model

Each entity becomes a relation with a primary key; each 1:N becomes a foreign key.



Relational schema

Concrete types, NOT NULL, CHECK and FOREIGN KEY, written in DBMS syntax.

KEY TAKEAWAYS

Data Modeling and Database Design

- 1 Data modeling builds a shared, precise vocabulary before any technology commitment is made.
- 2 Conceptual model (technology-independent) → logical model (relational structure, DBMS-independent) → relational schema (concrete types, constraints, syntax).
- 3 Relational database design is judged by correctness, non-redundancy, and robustness under real use — not merely “does the SQL compile.”
- 4 Mapping conceptual → logical: entity → relation, attribute → column, relationship → foreign key or new relation depending on cardinality.
- 5 A syntactically correct schema can still be a poor design — always check redundancy, anomalies, keys, integrity, growth, and security.